ใบงานที่ 5

ชื่อบทเรียน การทดลองวงจร BCD Arithmetic Unit

จุดประสงค์การสอน ปฏิบัติการทดลองวงจร BCD Arithmetic Unit

ออกแบบวงจร BCD Arithmetic Unit

ทดสอบวงจร BCD Arithmetic Unit

1. ทฤษฎี

การออกแบบวงจร BCD Arithmetic Unit

การออกแบบวงจร BCD Adder/Sub Tractor ฐานสิบ 3 หลัก

2. จงออกแบบวงจรต่อไปนี้

แสดงขั้นตอนการออกแบบวงจร BCD Adder/Sub ฐานสิบ 3 หลัก คือ หลักหน่วย , หลักสิบและหลักร้อย สามารถใช้ MSI IC ได้

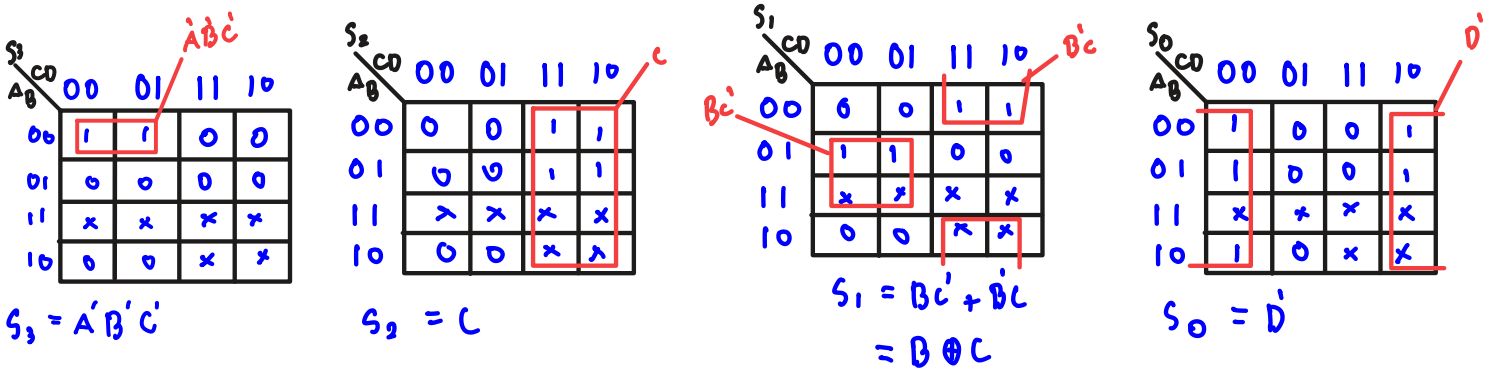
3. ผลการทดลอง

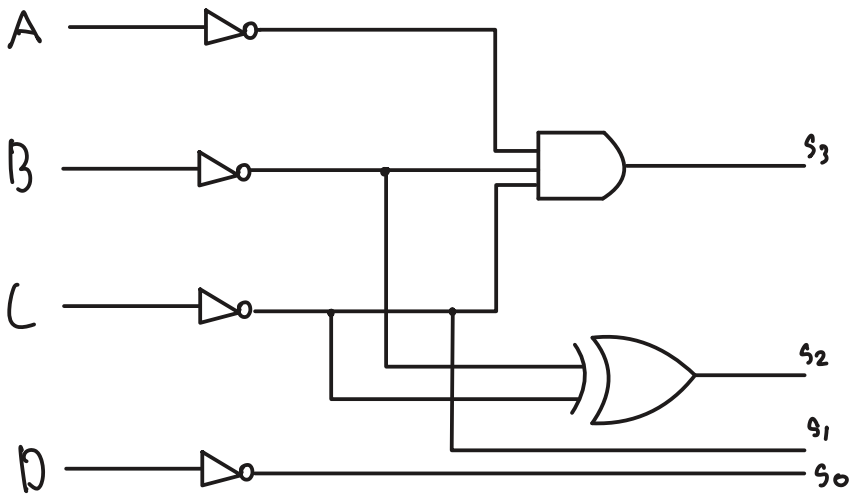
4. สรุปผลการทดลอง

5. คำถามหลังการทดลอง

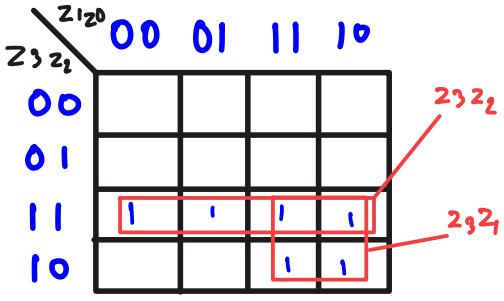
1. บอกหลักการแปลง binary adder มาสร้างเป็น BCD adder
2. บอกหลักการทำวงจรBCDบวกให้เป็นวงจรลบBCD
3. ถ้าต้องการออกแบบวงจรBCDบวกและลบเลขขนาด 5 หลัก จะต้องออกแบบอย่างไร

Decimal N	BCD ABCD N	Bitwise Complement 15 - N	+	Add 1010 10	=	Result 25 - N 16 + N'	Ignore carry-out = 9s Complement N'
0	0000	1111		1010		11001	1001
1	0001	1110		1010		11000	1000
2	0010	1101		1010		10111	0111
3	0011	1100		1010		10110	0110
4	0100	1011		1010		10101	0101
5	0101	1010		1010		10100	0100
6	0110	1001		1010		10011	0011
7	0111	1000		1010		10010	0010
8	1000	0111		1010		10001	0001
9	1001	0110		1010		10000	0000





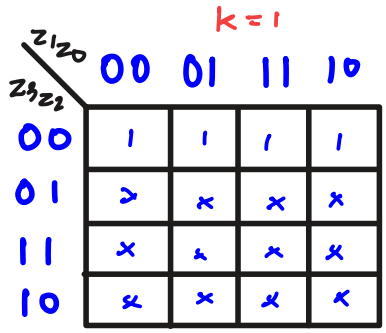
Decimal	Binary Sum						BCD Sum					BCD Sum	Weights
Sum	K	Z3	Z2	Z1	Z0		C	S3	S2	S1	S0	10	1
0	0	0	0	0	0	+0 =	0	0	0	0	0	0	0
1	0	0	0	0	1		0	0	0	0	1	0	1
2	0	0	0	1	0		0	0	0	1	0	0	2
3	0	0	0	1	1		0	0	0	1	1	0	3
4	0	0	1	0	0		0	0	1	0	0	0	4
5	0	0	1	0	1		0	0	1	0	1	0	5
6	0	0	1	1	0		0	0	1	1	0	0	6
7	0	0	1	1	1		0	0	1	1	1	0	7
8	0	1	0	0	0		0	1	0	0	0	0	8
9	0	1	0	0	1		0	1	0	0	1	0	9
10	0	1	0	1	0	+6 =	1	0	0	0	0	1	0
11	0	1	0	1	1		1	0	0	0	1	1	1
12	0	1	1	0	0		1	0	0	1	0	1	2
13	0	1	1	0	1		1	0	0	1	1	1	3
14	0	1	1	1	0		1	0	1	0	0	1	4
15	0	1	1	1	1		1	0	1	0	1	1	5
16	1	0	0	0	0		1	0	1	1	0	1	6
17	1	0	0	0	1		1	0	1	1	1	1	7
18	1	0	0	1	0		1	1	0	0	0	1	8
19	1	0	0	1	1		1	1	0	0	1	1	9

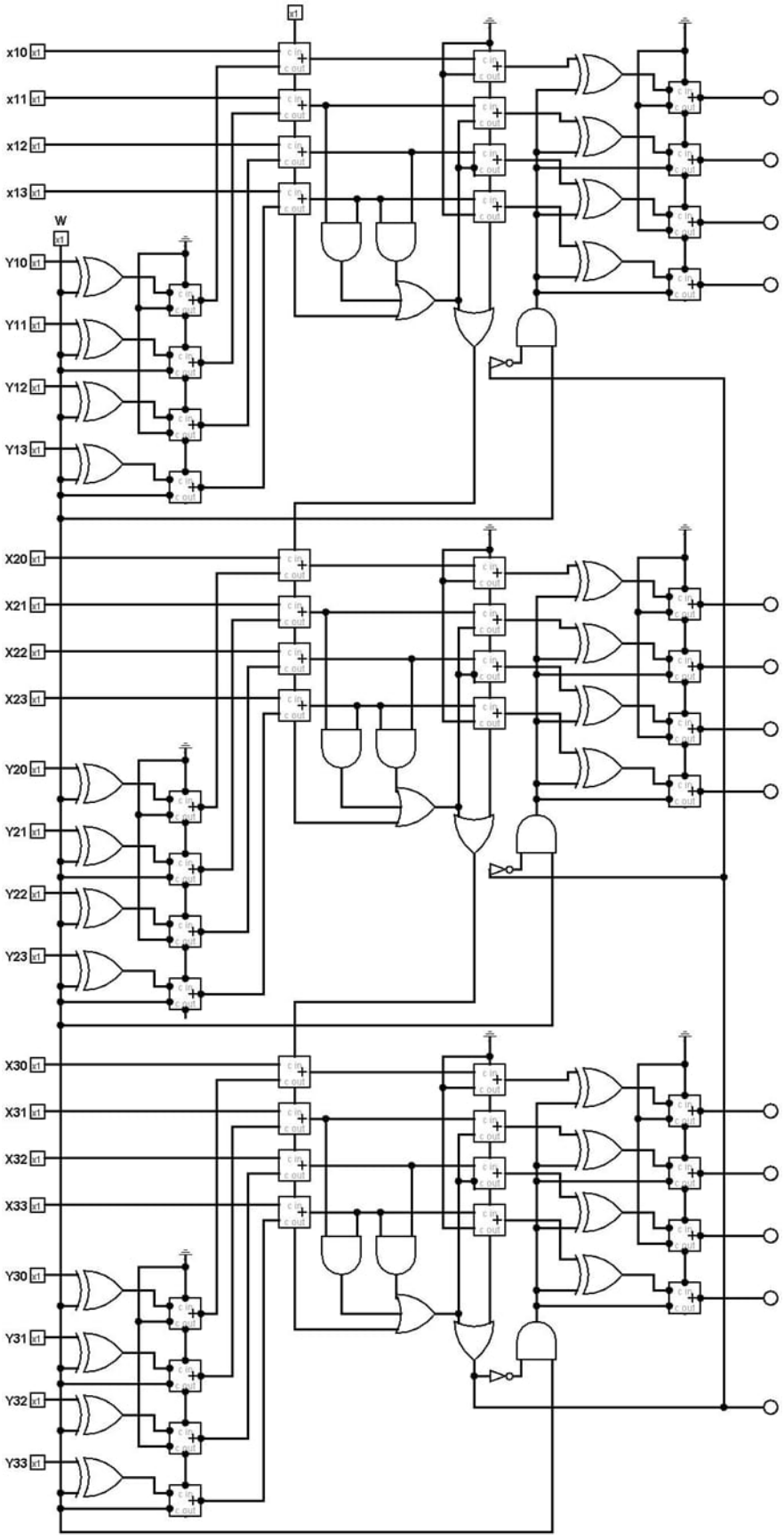


จะได้ฟังก์ชัน ของ C_0 ตัวนี้

$C_0 = k + z_3z_1 + z_3z_2$

และเมื่อ $C_0 = 1$ เราจะมอก 2 ด้วย 0110




x10
x11
x12
x13
W
Y10
Y11
Y12
Y13
X20
X21
X22
X23
Y20
Y21
Y22
Y23
X30
X31
X32
X33
Y30
Y31
Y32
Y33

BCD_ALU_1Digit.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity BCD_ALU_1Digit is
    Port (
        X    : in  STD_LOGIC_VECTOR(3 downto 0);
        Y    : in  STD_LOGIC_VECTOR(3 downto 0);
        W    : in  STD_LOGIC;  -- W=0 สำหรับบวก, W=1 สำหรับลบ
        Cin  : in  STD_LOGIC;
        S    : out STD_LOGIC_VECTOR(3 downto 0);
        Cout : out STD_LOGIC
    );
end BCD_ALU_1Digit;

architecture Dataflow of BCD_ALU_1Digit is
    signal Y_comp : STD_LOGIC_VECTOR(3 downto 0);
    signal Z      : STD_LOGIC_VECTOR(4 downto 0); -- เผื่อบิต Overflow
    signal K      : STD_LOGIC;
    signal S_corr : STD_LOGIC_VECTOR(4 downto 0);
    signal Cin_ext : UNSIGNED(4 downto 0);
begin
    -- 1. วงจรปรับแต่งอินพุต Y (ชุดเกตด้านซ้ายสุดในภาพ)
    -- ทำ 9's Complement เมื่อมีคำสั่งลบ (W=1)
    Y_comp <= Y when W = '0' else
              STD_LOGIC_VECTOR(to_unsigned(9, 4) - unsigned(Y));
```

```
    -- 2. วงจรบวกหลัก (แถว Full Adder ตรงกลาง)
    -- นำ X มาบวกกับ Y ที่ปรับแต่งแล้ว และตัวทดเข้า
    Cin_ext <= to_unsigned(1, 5) when Cin = '1' else (others => '0');
    Z <= STD_LOGIC_VECTOR(resize(unsigned(X), 5) +
                          resize(unsigned(Y_comp), 5) + Cin_ext);

    -- 3. วงจรตรวจสอบเงื่อนไข BCD (ชุดเกต AND/OR ตรงกลาง)
    -- K จะเป็น 1 เมื่อผลบวกเกิน 9 หรือมีตัวทดล้นออกไป
    K <= Z(4) or (Z(3) and Z(2)) or (Z(3) and Z(1));

    -- 4. วงจรปรับแก้ผลลัพธ์ (แถว Full Adder ขวาสุด)
    -- หาก K=1 จะทำการบวกเลข 6 (0110) เข้าไปเพื่อปรับให้เป็น BCD ที่ถูกต้อง
    S_corr <= STD_LOGIC_VECTOR(unsigned(Z) + to_unsigned(6, 5))
              when K = '1' else Z;

    -- จ่ายสัญญาณออก
    S <= S_corr(3 downto 0);
    Cout <= K;

end Dataflow;
```

tb_BCD_ALU_1Digit.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_BCD_ALU_1Digit is
end tb_BCD_ALU_1Digit;

architecture behavior of tb_BCD_ALU_1Digit is
   -- สร้างสัญญาณ (Signals) สำหรับจำลองการป้อนค่า
   signal X_tb   : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
   signal Y_tb   : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
   signal W_tb   : STD_LOGIC := '0';
   signal Cin_tb : STD_LOGIC := '0';

   -- สร้างสัญญาณสำหรับรับค่าผลลัพธ์
   signal S_tb   : STD_LOGIC_VECTOR(3 downto 0);
   signal Cout_tb: STD_LOGIC;

begin
   -- เชื่อมต่อพอร์ตเข้ากับสัญญาณทดสอบ (Instantiate UUT)
   uut: entity work.BCD_ALU_1Digit PORT MAP (
      X => X_tb,
      Y => Y_tb,
      W => W_tb,
      Cin => Cin_tb,
      S => S_tb,
      Cout => Cout_tb
   );
```

```
-- กระบวนการป้อนค่าทดสอบ (Stimulus Process)
stim_proc: process
begin
   -- กรณีที่ 1: การบวกปกติแบบไม่มีตัวทด (3 + 4 = 7)
   -- คาดหวัง: S_tb = "0111" (7), Cout_tb = '0'
   X_tb <= "0011"; Y_tb <= "0100"; W_tb <= '0'; Cin_tb <= '0';
   wait for 20 ns;
   assert S_tb = "0111" and Cout_tb = '0'
      report "3 + 4 should produce 7 with no carry" severity error;

   -- กรณีที่ 2: การบวกที่ผลลัพธ์เกิน 9 ต้องเกิด BCD Correction (5 + 7 = 12)
   -- คาดหวัง: S_tb = "0010" (2), Cout_tb = '1'
   X_tb <= "0101"; Y_tb <= "0111"; W_tb <= '0'; Cin_tb <= '0';
   wait for 20 ns;
   assert S_tb = "0010" and Cout_tb = '1'
      report "5 + 7 should produce 2 with carry" severity error;

   -- กรณีที่ 3: การบวกแบบมีตัวทดเข้า (8 + 1 + Cin(1) = 10)
   -- คาดหวัง: S_tb = "0000" (0), Cout_tb = '1'
   X_tb <= "1000"; Y_tb <= "0001"; W_tb <= '0'; Cin_tb <= '1';
   wait for 20 ns;
   assert S_tb = "0000" and Cout_tb = '1'
      report "8 + 1 + carry-in should produce 0 with carry" severity error;

   -- กรณีที่ 4: การลบปกติ (8 - 3 = 5)
   -- หมายเหตุ: การลบต้องป้อน W=1 และวงจรต้องการ Cin=1 เพื่อทำ 10's complement ให้สมบูรณ์
   -- คาดหวัง: S_tb = "0101" (5), Cout_tb = '1' (Cout=1 หมายถึงผลลัพธ์เป็นบวก)
   X_tb <= "1000"; Y_tb <= "0011"; W_tb <= '1'; Cin_tb <= '1';
```

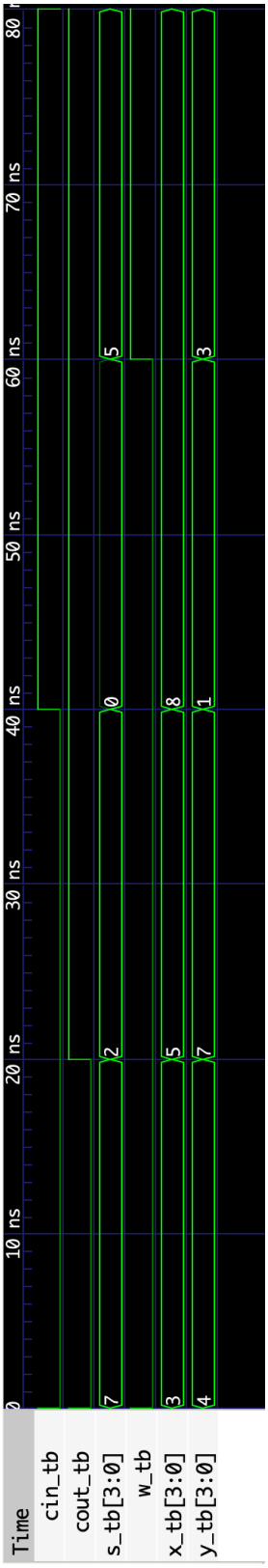
```
        wait for 20 ns;
        assert S_tb = "0101" and Cout_tb = '1'
            report "8 - 3 should produce 5 with carry" severity error;

        -- กรณีที่ 5: การลบแบบมีการยืมค่าจากหลักก่อนหน้า (7 - 4 - Borrow(1) = 2)
        -- การยืมค่าในวงจรบวก/ลบ ทำได้โดยให้ Cin=0 ตอนสั่งลบ
        -- คาดหวัง: S_tb = "0010" (2), Cout_tb = '1'
        X_tb <= "0111"; Y_tb <= "0100"; W_tb <= '1'; Cin_tb <= '0';
        wait for 20 ns;
        assert S_tb = "0010" and Cout_tb = '1'
            report "7 - 4 - borrow-in should produce 2 with carry" severity error;

        -- หยุดการทำงาน
        wait;
    end process;

end behavior;
```

ผลลัพธ์


Time
cin_tb
cout_tb
s_tb[3:0]
w_tb
x_tb[3:0]
y_tb[3:0]
0
10 ns
20 ns
30 ns
40 ns
50 ns
60 ns
70 ns
80
7
2
0
5
3
5
8
4
7
1
3